OptBin: A Practical Storage and Loading Tool for Sparse Optimization Models

Version 0.2.0rc4 – 30 October 2026

Abstract

OptBin converts supported continuous MPS and CBF models into compact lossless blocks and loads them through a validating C reader. This report describes the experimental tool, installation, verification and observed storage/loading trade-offs. Its practical purpose is repeatable numerical model distribution and loading, using established compression methods.

1 Implemented scope and contracts

The package provides MPS-to-OptBin and CBF-to-OptBin converters, a C11 reader, block traversal, an inspector, Python bindings, and a legacy BQP-to-v2 migration utility. Make and CMake builds are available. The core requires zlib and uses the optional Zstandard runtime for compressed blocks; it has no HDF5, CUDA or solver dependency. The current validated platform is 64-bit little-endian Linux x86-64. The v2 wire schema and C ABI remain experimental.

The bounded profile preserves LP/QP/QCQP objectives, variable/row bounds and native quadratic constraints. MPS extensions use producer-specific section vocabularies [1, 2]. QCQP conversion requires an explicit producer dialect. Gurobi 13.0.3 and MOSEK 11.2.7 reader regressions verify the implemented `QCMATRIX`/`QSECTION` scaling. CPLEX QCQP conventions are not validated. The continuous CBF 1–3 subset [3] retains scalar cones, original power parameters, PSD variables and sparse PSD operators. Integrality, change sequences and unknown features fail explicitly. Storage support is broader than the optional SCS/Clarabel and Gurobi/SCIP adapter coverage. Full details and parser exclusions are in `README.md` and `docs/format-v2.md`.

Text conversion produces a canonical binary64 numerical model; it may sort coordinates or sum duplicate MPS coefficients. Losslessness refers to exact reconstruction of that model, including supported signed zeros, rather than the original text, comments or names. Value blocks select raw, constant, exact small-integer, exact binary32 or dictionary representations. Integer blocks use raw or delta/zigzag encodings, followed by optional Zstandard [4]. These are established techniques integrated for this tool, not a new compressor.

Complete loading validates checksums, array coverage, sparse ordering, shapes, and cross-array model relationships. Block traversal validates framing, CRCs and coverage with bounded scratch, but does not establish all cross-array invariants. CRCs detect accidental corruption and are not authentication. Conversion uses exclusive publication and never overwrites an existing file.

2 Build, installation and use

Native tools require a C11/C++17 toolchain, CMake 3.16+ and zlib development files. The Zstandard runtime is needed to read Zstd-compressed blocks; write uncompressed blocks with `-zstd 0` when it is unavailable. From unpacked source, a typical local installation is:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -DBUILD_SHARED_LIBS=ON
cmake --build build -j4
ctest --test-dir build --output-on-failure
cmake --install build --prefix "$HOME/.local/optbin"
~/.local/optbin/bin/mps2optb examples/qp.mps /tmp/qp.optb
~/.local/optbin/bin/optb_inspect /tmp/qp.optb --load
```

Installed CMake consumers link `OptBin::optbin`; an external example is provided under `examples/consumer/`. The installed prefix can be relocated. The public C entry point is `optb_load`; release successful models with `optb_free`. Its limit bounds decoded-array bytes, not process RSS.

The Python distribution requires CPython 3.10+. Source installation needs a C compiler, Python headers and zlib development files. Pip builds a bundled C reader, so the installed package works outside its source checkout:

```
python3 -m venv .venv
.venv/bin/python -m pip install .
.venv/bin/optbin-info /tmp/qp.optb --limit 8589934592
```

The base binding has no NumPy/SciPy dependency. Optional extras enable solver adapters and the Python HDF5 experiment; standalone converters and the C development library use the native installation. Python source/wheel and relocated native installs are exercised by `scripts/check_install.py`. The wheel has a local Linux platform tag; no manylinux or cross-distribution binary compatibility is claimed. CI tests CPython 3.10/3.12/3.14, optional frontends, independent-reader contracts, UBSan and compilation of this note. Hosted results are available in GitHub Actions.

3 Verification and large sparse indices

Complete numerical comparisons with independent source readers pass on 22 continuous models (13 MPS LP/QP/QCQP models read independently by Gurobi and nine CBFs by both MOSEK and the original CBLIB parser). All 6 selected integer CBFs are rejected explicitly. The archived solver study contains 55 source/native runs; all 13 paired Gurobi cases return optimal status. Some conic/SDP runs are inaccurate or inconclusive and two SCIP runs reach their time limits. All eight large source BQP payload SHA-256 hashes match their existing manifest.

An additional JuMP 1.31.2/MOI 1.53.0 source-reader check matches all 22 real models and 15 fixture/configuration cases after explicit function-in-set normalization. Five targeted MPS fixtures retain reader discrepancies, adjudicated against MOSEK or Gurobi; one generalized power cone is unsupported by MOI. These six cases are not agreement passes. Details and pinned reproduction are in `docs/jump-validation.md`.

The checks include 22 core/legacy groups, four JuMP-comparison harness groups, 14 optional groups, five Gurobi-reference groups, four MOSEK-reference groups and four gzip-benchmark groups, including malformed inputs, budgets, exclusive output, exact codecs and independently constructed raw files. Core and HDF5 checks also passed UBSan. Release installation checks additionally exercise fresh virtual environments, known numerical arrays, CLI error handling, a relocated native prefix and an external CMake consumer. Python and inspector limits outside $[0, 2^{64} - 1]$ are rejected; Python paths containing null bytes fail before entering C. Model-data comparison is distinct from solver success; inaccurate and inconclusive outcomes remain visible in the archived reports.

For CSC with n columns, N stored entries, 64-bit pointers, row-index width w and binary64 values, the decoded sparse-array footprint is $8(n+1)+(w+8)N$ bytes. Pointer width depends on entry count; row-index width depends on row IDs. They must be selected independently. A separate full load of `prod_200_500_02` reconstructs 4,443,349,500 entries exactly against its BQP source, retaining 32-bit row indices and 64-bit pointers. This verifies an entry count beyond 2^{32}; it is a correctness-only check, not a repeated timing result.

4 Compressed storage and complete loading

The practical comparison uses MPS.gz or CBLIB CBF.gz, externally gzipped raw HDF5 arrays [5] (`raw.h5.gz`), default OptBin and externally gzipped default OptBin (`OptBin.gz`). Raw HDF5 contains unencoded binary64 values, 32/64-bit indices, metadata and block checksums, with no dataset compression or shuffle filter. External gzip is applied to the entire file. HDF5 is 1.14.6.

MPS.gz timing uses public `gurobipy.read` from Gurobi 13.0.3 [7], ending with its solver model ready. Environment startup and subsequent reference extraction are measured separately. CBF.gz uses public MOSEK 11.2.7 `Task.readdata` [8], ending with its solver Task ready. Full conic functions, domains and PSD trace coefficients agree exactly with OptBin after explicit coordinate normalization. All public-API extraction and comparison occur outside read timing. The binary paths return numeric arrays that pass full `optb_validate`; binary checksums are also verified. These are practical public loader endpoints with different internal model representations. No optimization, presolve, GPU upload or model-file output is timed. Numerical comparisons occur after the timer and RSS snapshot. Files are warm; trials use fresh processes in shuffled rounds. Only large-case RSS is interpreted: small subprocess peaks can retain a pre-exec parent watermark, rather than reflect minimum reader memory.

The binary gzip paths decompress to a seekable temporary file on the same filesystem before native loading, with both steps included in the timer. `OptBin.gz` is a transport wrapper: the C API itself expects the expanded `.optb` file. Generated gzip files use level 6; existing CBLIB gzip settings are unknown. Gzip creation times are recorded separately as preparation; they are not charged to repeated loading. The default OptBin writer already uses block encodings and Zstandard. This compares practical representations, not isolated compressors or all possible HDF5 reader implementations.

All 22 real continuous models pass independent source-model comparisons; the three binary representations reconstruct identical numerical arrays. The table shows eight examples spanning LP, QP, QCQP, SOCP, exponential cones and PSD data, measured on 12th Gen Intel(R) Core(TM) i9-12900.

	Text.gz		raw.h5.gz		OptBin		OptBin.gz	
Model	KiB	ms	KiB	ms	KiB	ms	KiB	ms
afiro	0.60	2.242	1.68	1.849	1.98	0.209	0.91	0.338
AUG2D	401.58	15.887	130.64	5.047	2.13	2.202	0.87	2.325
AUG3DCQP	62.93	5.073	29.67	2.966	2.13	0.583	0.91	0.692
QPLIB_2676	71.59	5.852	17.86	3.020	7.59	0.614	5.94	0.807
nql30	71.11	4.888	38.32	3.060	21.54	0.837	18.83	1.051
nb_L2	2194.08	54.388	760.63	12.660	613.99	6.888	607.30	9.358
LogExpCR-n20-m400	109.96	4.530	72.67	2.330	66.16	0.601	63.78	1.060
infeas_clean_10_10_1	1.30	0.956	2.99	1.844	4.09	0.264	1.39	0.425

Table 1: Stored KiB (2^{10} bytes), including metadata, and complete loading medians in milliseconds (seven shuffled fresh-process trials per format). Text.gz is MPS.gz read by Gurobi in the first four rows and original CBLIB CBF.gz read by MOSEK in the last four. Decompression and binary temporary-file staging are included.

On the 13 MPS models, default OptBin loads 6.42–10.75 times as fast as public gurobipy.read; OptBin.gz loads 6.13–7.74 times as fast. On the nine CBF cases, default OptBin loads 3.62–8.00 times as fast as MOSEK Task.readdata; OptBin.gz loads 2.25–6.88 times as fast. Across all 22 cases it loads 1.84–8.87 times as fast as the raw.h5.gz temporary-file path. The highly compressible AUG cases are favorable examples. Default OptBin is larger than Text.gz on 5 of 22 models; OptBin.gz is larger on 2, afiro and the tiny clean PSD case. Outer gzip sometimes reduces OptBin metadata substantially, but also adds a decompression/staging step. Small-case results include reader initialization and metadata costs and are not large-model speedup claims.

Large LP files

Gurobi reads both original large MPS.gz files; complete public-API LP comparisons match their OptBin numerical models. A separate post-timing metadata audit confirms continuous single-objective LP profiles, unchanged file hashes and Gurobi fingerprints. The binary representations also pass decoded-array byte comparisons. These files retain previously migrated BQP data. The shared host uses INTEL(R) XEON(R) GOLD 6548Y+. prod has 16,361,200 rows, 64,560,800 columns and 2,310,559,600 matrix nonzeros; mcf has 5,525,100 rows, 502,500,100 columns and 1,007,500,100 matrix nonzeros. Two case jobs run concurrently; each loads one format at a time in shuffled rounds. The prod gzip files use GNU gzip with one compression thread; mcf uses pigz 2.8 with 16 compression threads, both at level 6. Native decoding is serial; Gurobi uses Threads=1. Environment startup and reference extraction are excluded from Gurobi read time.

Representation	prod GiB	prod seconds	mcf GiB	mcf seconds
MPS.gz (Gurobi)	7.294	342.27	10.553	386.89
raw.h5 (uncompressed)	27.992	–	26.319	–
raw.h5.gz	0.533	101.39	4.301	117.72
OptBin	0.441	53.13	4.460	53.59
OptBin.gz	0.421	55.41	4.314	73.37

Table 2: Large LP storage and complete loading. Times are medians of three shuffled fresh-process trials. Raw .h5 has storage measurements only. prod is `prod_200_400_02`; mcf is `mcf_5000_100_1000`. GiB means 2^{30} bytes. Gurobi returns its solver model; the binary paths return validated numerical arrays.

Before gzip, raw HDF5 occupies 27.99 and 26.32 GiB (prod/mcf), 63.4 and 5.9 times default OptBin. The tested raw.h5.gz loader needs a temporary file of that expanded size; default OptBin reads its internally compressed file directly. These savings concern stored files and temporary disk space; decoded RAM is essentially unchanged. HDF5 also supports internal dataset compression, which this raw-array baseline does not use.

For `mcf_5000_100_1000`, raw.h5.gz is smaller than both OptBin representations; stored size and loading time favor different choices. Default OptBin loads 6.44 and 7.22 times as fast as Gurobi on these cases; OptBin.gz loads 6.18 and 5.27 times as fast. Creating raw.h5.gz from the existing raw HDF5 files took 208.6 and 48.5 seconds; creating OptBin.gz from the existing OptBin files took 14.8 and 8.3 seconds. These are gzip-only preparation costs, not source-to-binary conversion times.

OptBin peak process RSS is 28.00 and 26.32 GiB; the other binary paths are within 0.01 GiB. Gurobi read-phase RSS is 57.67 and 70.41 GiB. The latter includes the vendor model and its metadata, so this is not a pure container-memory comparison. Expanded numeric arrays dominate memory. RSS excludes filesystem page cache, expanded gzip temporaries and later solver allocations. Full verification temporarily holds additional model and array copies. Binary gzip staging needs temporary space equal to the uncompressed file size.

The prod matrix contains 2,310,559,600 entries. Gurobi reads it successfully; in 13.0.3, `getA()` then fails while querying the legacy 32-bit nonzero count. Full verification uses public row-only model copies below the query limit, with complete row/nonzero coverage. This extraction work is outside read timing and its RSS snapshot.

5 Limits, provenance and reproducibility

Full model loading allocates expanded arrays. The MPS converter uses temporary section spools; arbitrary-coordinate CBF conversion currently sorts in memory. There is no v2 GPU adapter, direct storage-to-GPU access or compressed solver computation. Cold-I/O studies, more large QP/QCQP/PSD workloads, complete original-coordinate infeasibility certificates and broader portability remain outside this release.

Source collections retain their identifiers, attribution and licenses: Oliver Hinder's large LPs[1], QPLIB[2] and CBLIB[3]. The software is MIT licensed; external datasets are not bundled or relicensed. Benchmark datasets must be obtained separately; source identifiers and verified file hashes are retained in the benchmark reports.

Full tables, per-trial data and reproduction commands are linked from `docs/gzip-comparison.md`. Installation and feature limits are in `docs/release.md` and `docs/validation.md`. The report builder, `scripts/build_technical_report.py`, updates marked regions from the saved results and records their SHA-256 hashes in `docs/report-evidence.json`. The independent JuMP/MOI checks use the documented file-format API [6]; the saved version pins and known discrepancies remain part of the reproducibility record. Gurobi source checks and timing contracts are documented in `docs/gurobi-reference.md`; the corresponding CBF check and reference-helper limits are in `docs/mosek-reference.md`.

References

[1] Gurobi Optimization. *Model File Formats*, MPS reference. Official documentation.

[2] MOSEK. *The MPS File Format*. Official documentation.

[3] MOSEK. *The Conic Benchmark Format*, technical reference. Official specification.

[4] Y. Collet and M. Kucherawy (ed.). *Zstandard Compression and the 'application/zstd' Media Type*. RFC 8878, 2021. RFC 8878.

[5] The HDF Group. *HDF5 File Format Specification*, version 3.0. Official specification.

[6] JuMP developers. *MathOptInterface FileFormats Overview*. Official documentation.

[7] Gurobi Optimization. *Python Global Functions: read; Model Queries: getA and getAttr*. Read reference; model reference.

[8] MOSEK. *Supported File Formats; Optimizer Task API*. CBF and compressed inputs; public Task reference.

[1] `https://www.oliverhinder.com/large-scale-lp-problems`
[2] `https://qplib.zib.de/`
[3] `https://cblib.zib.de/`